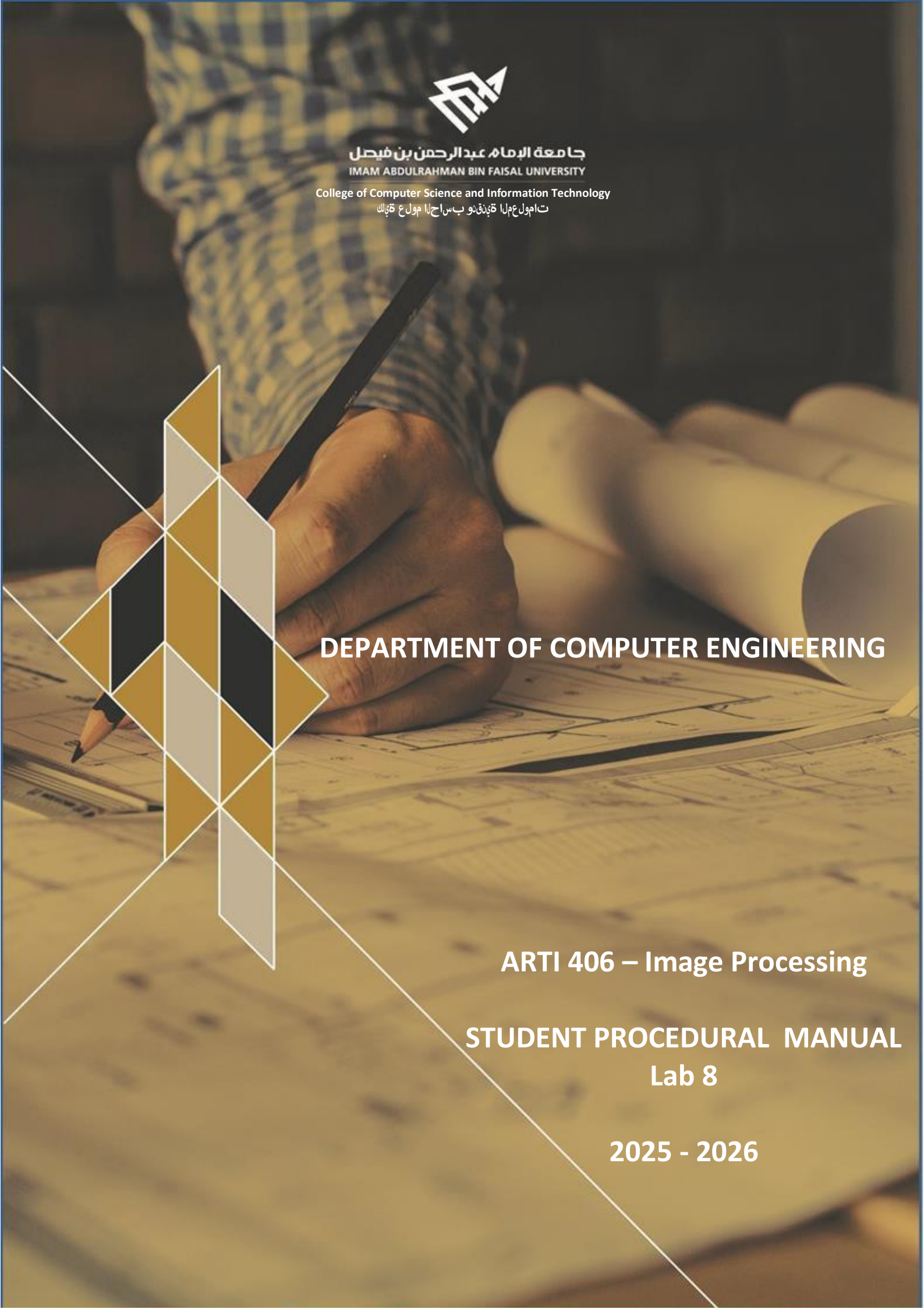


جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
College of Computer Science and Information Technology
كلية علوم الحاسب وتقنية المعلومات

DEPARTMENT OF COMPUTER ENGINEERING

ARTI 406 – Image Processing

STUDENT PROCEDURAL MANUAL
Lab 8

2025 - 2026

IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Lab 8 – Answers: Edge Detection

Objective: Apply and compare edge detection methods using Python and OpenCV, and investigate how parameter changes affect the detected edges.

Selected image: Butterfly.jpg (800 × 532 pixels). I used a new image instead of kid1.jpg from the original notebook. I chose a butterfly because it has strong edges (wing outline, black veins, white spots) and also weak, fine details (wing texture, leaves and flowers in the background), which makes it good for comparing the methods.

Task 1: Apply Edge Detection Methods

First, the image is loaded and converted to grayscale. For Sobel and Prewitt, a 5×5 Gaussian blur is applied before taking the derivatives to reduce noise. The gradient is computed in the x and y directions and combined using the magnitude $\sqrt{(Gx^2 + Gy^2)}$. Canny is applied directly on the grayscale image because it already includes smoothing inside the algorithm.

```
img = cv2.imread("Butterfly.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Sobel
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel_final = np.uint8(np.clip(cv2.magnitude(sobelx, sobely), 0, 255))

# Prewitt
kx = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float64)
ky = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float64)
px = cv2.filter2D(blurred, cv2.CV_64F, kx)
py = cv2.filter2D(blurred, cv2.CV_64F, ky)
prewitt_final = np.uint8(np.clip(cv2.magnitude(px, py), 0, 255))

# Canny
canny_edges = cv2.Canny(gray, 100, 200)
```

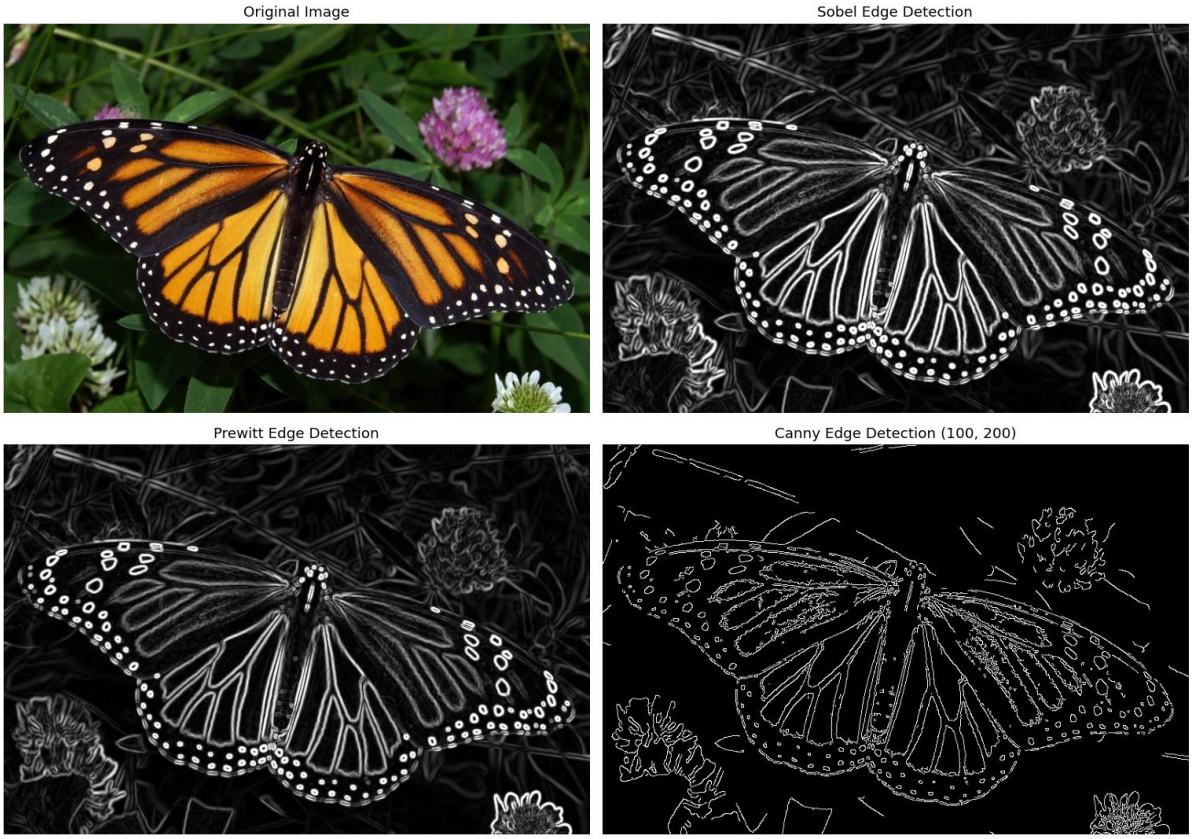


Figure 1: Original image and the Sobel, Prewitt and Canny edge maps

Comparison of the three methods

Sobel: The edges are thick and bright. The wing outline, the black veins and the white spots are very clear. Because the Sobel kernel gives a weight of 2 to the centre row/column, it smooths a little while differentiating, so it handles noise better than Prewitt.

Prewitt: The result looks very similar to Sobel, but the edges are slightly weaker (darker) because all rows of the kernel have the same weight. The background leaves and grass still appear as soft gray edges.

Canny: The edges are thin (about one pixel wide), binary (black or white) and well connected. Weak responses from the background are mostly removed by non-maximum suppression and hysteresis thresholding, so Canny gives the cleanest and most precise edge map of the three.

Task 2: Investigate the Effect of Canny Thresholds

Canny edge detection was applied on the same grayscale image with three different threshold pairs. I also counted the number of edge (white) pixels in each result using np.count_nonzero() so the results can be compared with numbers and not only by eye.

```
thresholds = [(30, 90), (50, 150), (100, 200)]
for i, (low, high) in enumerate(thresholds):
    edges = cv2.Canny(gray, low, high)
    print(low, high, np.count_nonzero(edges))
    plt.subplot(1, 3, i + 1)
    plt.imshow(edges, cmap="gray")
    plt.title(f"Canny: Low = {low}, High = {high}")
```

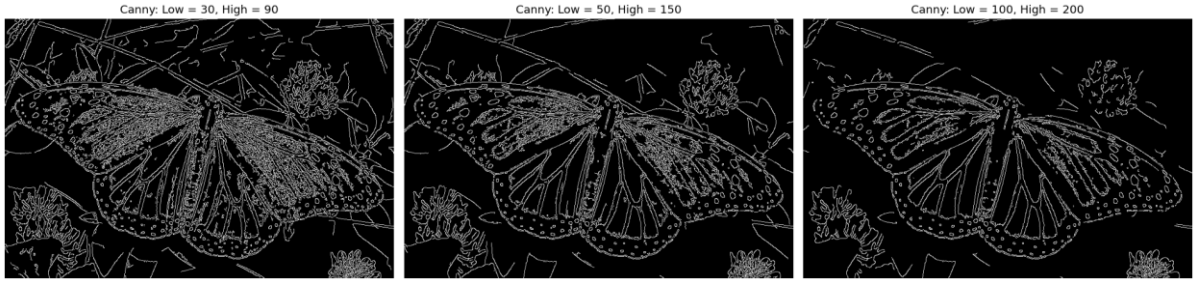


Figure 2: Canny edge maps with the three threshold pairs (side by side)

Low	High	Edge pixels	Observation
30	90	53,410	Most edges; a lot of fine texture and background noise
50	150	38,430	Balanced; main structure clear, less clutter
100	200	26,682	Fewest edges; only strong edges, some details lost

Explanation

In Canny, the **high threshold** decides which pixels are "strong" edges (they are always kept), and the **low threshold** decides which "weak" pixels can be kept if they are connected to a strong edge (hysteresis). Any pixel with a gradient below the low threshold is removed.

Low = 30, High = 90: Because the thresholds are low, even small changes in intensity are accepted as edges. This gave the highest number of edge pixels (53,410). Many fine details are visible, like the texture inside the orange parts of the wings and many lines from the leaves and grass, but the image looks cluttered and noisy.

Low = 50, High = 150: The number of edge pixels dropped to 38,430 (about 28% less). Most of the background texture disappeared, while the wing outline, veins, white spots and flowers are still clear. This gives a good balance between detail and noise.

Low = 100, High = 200: This gave the fewest edge pixels (26,682, about half of the first case). Only the strongest edges remain, such as the butterfly outline and the main veins. The image is very clean, but weak edges and small details are lost and some contours become broken.

Conclusion: Increasing the thresholds reduces the number of detected edges. Low thresholds show more image details but also more noise and unwanted edges, while high thresholds remove noise but may also remove real weak edges. For this image, the pair (50, 150) gives the best result because it keeps the important structure of the butterfly without too much background noise.